Technical Requirements Document: Table Tennis League Management System

Oct 17, 2026 · @kelliosli

1. Overview and scope

This system runs a youth table tennis league end to end: season setup, team and roster management, seeding, scheduling, live scoring in the hall, ratings and public results. It must ship before the end of November 2026.

In scope: seasons, tours, stages and composite formats; 24 teams with 72 subteams (3 age groups) and 360+ players; lineups; hall operations; live scoring on tablets; Elo ratings; season ranking; public site in Russian, Kazakh and English.

Out of scope for the first release: the CSV import of external ratings (schema only, importer later), player accounts, payments, native mobile apps.

Item	Value
Team	2 backend engineers, 1 frontend engineer
Deadline	End of November 2026
Backend	Django, Django Channels (WebSockets)
Database	PostgreSQL
Hosting	VPS with automatic backups
Scale	about 1,000 concurrent viewers, about 15 concurrent matches, 12 tables by default
UI languages	Russian, Kazakh

Terms. *Age group* and *division* are synonyms (U13, U15, U17, one division each). A *team* is a club with 3 *subteams*, one per age group. A *team match* is up to 5 *singles games*; a singles game is best of 5 *sets*.

2. Domain rules

The rules below are requirements the backend must enforce, not UI conventions.

2.1 Competition structure

- A **season** has one or more **tours**, configured before the season starts. Each tour has its own **stages**.
- A stage has a **format** from a configurable set: round-robin groups, single elimination, or a composite (for example, round-robin groups of 24 subteams followed by 3 single-elimination brackets seeded by group placement). Composite formats are defined as data (an ordered list of

stage templates and the rule that maps stage-N placements to stage-N+1 seeds), not as hard-coded branches.
- Subteams are distributed into groups by seeding on subteam rating (section 2.5).
- Age category is determined by year of birth only. Players can drop out mid-tournament; a dropped player stays in history and past results, and cannot be put in new lineups.

2.2 Team match

A team match is 5 singles games, strictly gendered slots:

Slot	Game
1	1st boy vs 1st boy
2	1st girl vs 1st girl
3	2nd boy vs 2nd boy
4	2nd girl vs 2nd girl
5	3rd boy vs 3rd boy

- The trainer submits the lineup before the match. After submission it is locked. Each side's lineup is hidden from the other (and from the public) until both are submitted, then both are revealed at once.
- A valid lineup has at least 3 boys and 2 girls. An unfilled slot is an automatic loss of that singles game. A no-show team loses the match.
- Only the Main Judge (or an Organizer) may replace a player after lock, including a dropped-out player, even mid-match. Every replacement is audit-logged.
- The match ends when one team reaches 3 game wins; remaining games are cancelled and not played.

2.3 Singles game and scoring

- Best of 5 sets (first to 3). A set goes to 11 points, win by 2 (deuce continues indefinitely).
- The line judge decides who serves first. Serve alternates every 2 points, and every 1 point once the score reaches 10:10. Sides change after each set. The server indicator is derived from the point log, not stored separately.
- Score entry is point by point. Undo of points and sets is allowed for either player and is a logged event, not a delete.

2.4 Elo rating

Each player has an individual Elo rating, recalculated automatically after every finished singles game. Players without an external rating start at 1000. The K-factor is set per tournament before it starts; a change after start applies only to games not yet calculated, and each rating history row stores the K used.

```
E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, \qquad R_A' = R_A + K\,(S_A - E_A)
```

where S_A is 1 for a win and 0 for a loss. Automatic-loss games (empty slot) do not change ratings. *The 400 scale is the standard Elo constant; the source spec left the formula blank, so see open questions.*

2.5 Subteam rating and seeding

Subteam rating is the sum of the 5 lineup players' ratings and is used for seeding:

```
R_{subteam} = \sum_{i=1}^{5} R_i
```

Because lineups exist only per match, seeding before the first match needs a defined input. Proposal: use the subteam's declared **seeding roster** (top 5 eligible players by rating, 3 boys and 2 girls) as of the seeding freeze time, and snapshot the result in the stage record so later rating changes do not reshuffle a draw.

2.6 Season ranking

A subteam's placement in each tour is recorded. The season score aggregates those placements; the lowest total wins and the ranking is sorted ascending. Ties are broken by the head-to-head result between the tied subteams. Proposal for the aggregation: the sum of tour placements, with a per-season configurable weight per tour (default 1).

3. Roles and functional requirements

Five actors, enforced server-side on every REST call and WebSocket subscription; the UI only hides what the API would refuse.

Role	Created by	Can do	Cannot do
Organizer (several allowed)	Organizer	Everything: seasons, tours, stages, accounts, settings, all teams, all Main Judge actions	none
Main Judge (several allowed)	Organizer	Run the tournament: start matches (single and "start all"), override table allocation, replace players, manage all teams	Create seasons, create accounts, change tournament settings
Trainer	Organizer	Edit own team (all 3 subteams): players, rosters, lineups; view upcoming matches	Touch other teams, change a locked lineup
Line Judge	Organizer	Score matches on the one table assigned for that day: points, undo points and sets	Any other table or any admin action
Public viewer	none (no login)	Read matches, timetable, draws, team lists, rosters, live scores	Any write; see hidden lineups

Players have no accounts. A trainer may coach several subteams of the same team, so the trainer-to-subteam link is many-to-many but always within one team.

3.1 Functional requirements

Administration

- FR-1: Create and configure seasons, tours, stages and composite stage formats before a season starts; configuration locks at season start.
- FR-2: Create trainer, line judge, main judge and organizer accounts; reset passwords; deactivate accounts.
- FR-3: Tournament settings: K-factor, number of tables (default 12), rest time between a player's games, finals priority.

Teams and players

- FR-4: Trainers create and edit players (fields in section 5), up to 10 per subteam, no gender quota. Age group is computed from year of birth and validated on assignment.
- FR-5: Mark a player dropped out, with date and reason; keep all history.
- FR-6: First and personal trainer link to a trainer profile if one exists, otherwise store plain first and last name.

Draw and schedule

- FR-7: Generate groups and brackets by seeding from subteam rating; allow manual adjustment by Organizer before publish.
- FR-8: Auto-allocate matches to tables honoring rest time, no player in two games at once, and finals priority; Main Judge can override any assignment manually, with audit log.

Match day

- FR-9: Trainer submits a lineup that satisfies the slot rules; locked on submit; hidden until both sides submit, then revealed atomically.
- FR-10: Match lifecycle: scheduled, called, in_progress, finished. "Called" marks the match and highlights it on screens and public pages (the spoken announcement itself is human).
- FR-11: Main Judge starts a match by button, or starts all matches currently assigned to tables.
- FR-12: Line judge tablet shows player names, server indicator and live score; buttons add a point, undo a point, undo a set, timeout for either player.
- FR-13: Every score change is streamed live and persisted, and written to a full audit log (who, when, before, after).
- FR-14: Match ends at 3 game wins; the remaining games are cancelled automatically. No-show and empty-slot rules apply automatically.

Results

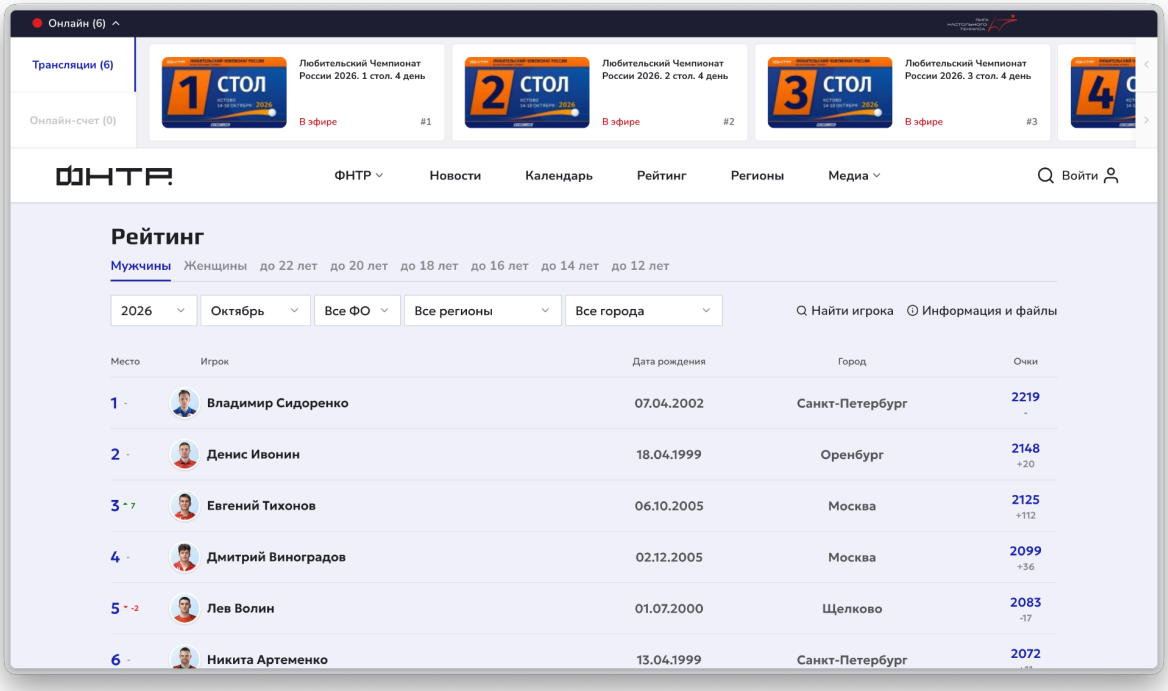
- FR-15: Elo recalculated after each finished singles game; rating history kept per game.
- FR-16: Tour placements, season score and season ranking computed automatically, including head-to-head tie-breaks.
- FR-17: Public site: matches, timetable, draws, team lists, rosters, live scoreboard, in Russian, Kazakh and English.
- FR-18 (later release): CSV import of initial ratings; the schema must already accommodate it.

URL Frontend Structure:

For everyone(without authentication)

‘/’ - main page, live matches in the header, table of the league, individual ranking (https://ttfr.ru/ - as example for rating and header)

‘/ranking’ - as in the below photo, with the individual rankings



‘/player/{id} - player individual page, best wins, rating history (https://ttfr.ru/sportsman/566 - graph of rating, matches history by the tournament)

‘/seasons/’ - dashboard with all the seasons as the cards

‘/season/{id}/’ - season page with the general info(tours, teams quantity)

‘/tournament/{id}/’ - information about the tournament(detailed),

Note: tournament is the one of the tours of the season

organizer and main judge(s) endpoint:

‘/tournament/{id}/matches’

‘/tournament/{id}/match/{id}’

‘tournament/{id}/draws/{id}’

‘/

linear judges endpoint:

‘/linear-judge’ - interface for the linear judge (buttons to add point, remove point)

Trainers:

‘/team’ - view the available teams to manage

‘/team/{id}’ - manage the subteam

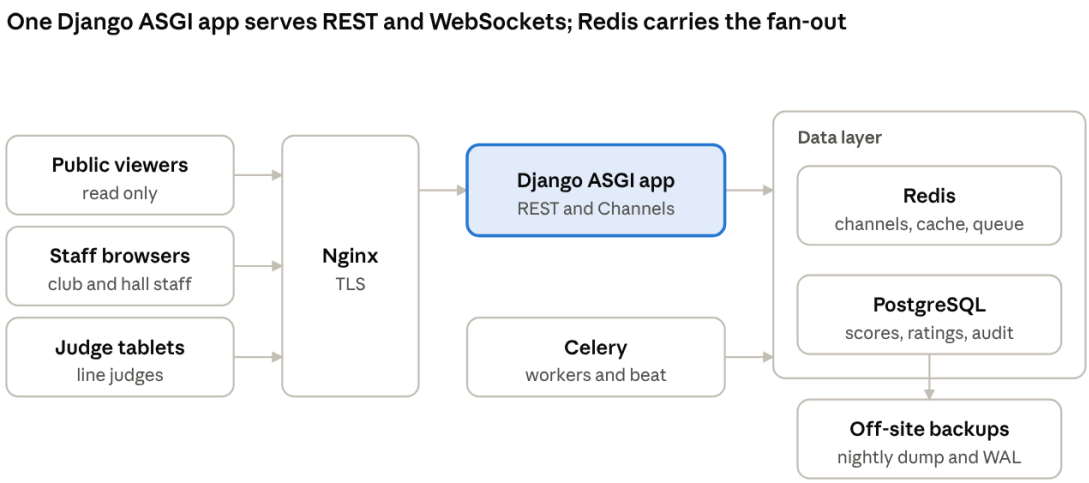
‘/subteam/{id}’ - add/select(if they are previously created) the 8-10 players

‘/subteam/{id}/tour/’ - edit the roster for tour(3 boys, 2 girls)

‘/

4. Architecture and stack

Recommendation: a single Django monolith served over ASGI, with React for the UI and Redis doing double duty as the Channels layer and the Celery broker. It keeps the moving parts few for a 3-person team with about 6 weeks, and every component is mainstream and well documented.



architecture · 3 client types, 5 server components

Clients reach one Nginx entry point; the Django app handles both HTTP and WebSockets, and both it and the Celery workers share Redis and PostgreSQL. Only PostgreSQL is backed up off-site.

Layer	Recommendation	Why	Main alternative and why not
Backend	Django 5 + Django REST Framework, Django Channels	Fixed by the brief; one codebase for API, admin and WebSockets	none
ASGI server	Uvicorn (or Daphne) behind Nginx	One process type serves HTTP and WebSockets; Nginx terminates TLS	Gunicorn + separate WS server: two stacks to run
Database	PostgreSQL 16	Fixed by the brief; transactions and constraints protect scoring and ratings	none
Frontend	React + TypeScript + Vite, TanStack Query, react-i18next, installable PWA	One engineer can build admin, trainer, judge and public views in one app; typed API client; PWA suits tablets	Next.js: adds a Node service to run; Django templates + HTMX: fast to start but live tablet and scoreboard UI gets awkward
UI kit	A component library (for example MUI or Mantine) with Russian/Kazakh/English locale packs	Saves weeks on forms, tables, dialogs	Hand-built components: too slow
Realtime transport	Channels over WebSocket, JSON messages, Redis channel layer	Matches the brief; groups per match fan out to viewers	Server-Sent Events: simpler, but tablets also send commands and benefit from one socket
Queue / background	Celery + Redis, Celery beat for schedules	Redis is already required; mature retries and scheduling	Postgres-backed queues (Procrastinate): one less service, smaller ecosystem; Django-Q2: lighter, less proven under load

Layer	Recommendation	Why	Main alternative and why not
Cache	Redis	Cached public read endpoints with short TTL, invalidated on result events	none
Files	Player photos on VPS disk or S3-compatible storage, thumbnails via Celery	Photos are small and few	CDN: not needed at this scale
Deployment	Docker Compose on the VPS: nginx, app (ASGI), celery worker, celery beat, postgres, redis	Reproducible, one command to update	Kubernetes: overkill
Backups	Nightly logical dump plus continuous WAL archiving (pgBackRest or WAL-G) to off-site object storage	Point-in-time recovery during a tournament day	Provider snapshots only: coarse, restore is slow
Observability	Sentry, structured logs, Prometheus + Grafana (or a hosted uptime check)	Live event needs fast failure detection	Logs only

What goes in the queue, and what does not. Elo recalculation and match-state changes run synchronously in the same database transaction as the game result, so scores and ratings can never disagree. Celery handles work that can lag by seconds: tour and season ranking recomputation, photo thumbnails, cache warming, the later CSV import, scheduled backup verification and email.

Frontend recommendation in one line: React SPA with a lean public view; server-side meta tags for the public pages if search or link previews turn out to matter (open question 6).

Sizing proposal for the VPS (to be load-tested): 4 vCPU, 8 GB RAM, SSD, with Postgres tuned for a small working set. The load is modest: about 15 matches produce a few events per second in total.

5. Data model

PostgreSQL, one schema, integer or UUID primary keys, `created_at` / `updated_at` on every table. The seed list from the brief is kept and extended below; field names are proposals.

5.1 People and teams

Table	Key fields	Notes
user	id, name, email (unique), password hash, role, is_active	Role: organizer, main_judge, trainer, line_judge. Subclasses as profile tables or a role column plus profile
trainer_profile	user (1:1), email	Linked from player.first_trainer and player.personal_trainer when the trainer exists
team	id, owner (trainer), name, region, created_at, updated_at	24 per season
subteam	id, team, age_group (U13, U15, U17)	Unique (team, age_group); 72 total
subteam_trainer	subteam, trainer	Many-to-many; the trainer must own or coach the parent team
player	id, first_name, last_name, subteam, gender, playing_hand, playing_style, date_of_birth, photo, sports_level, current_rating, status, external_rating_id	Age computed from birth year; age group must match subteam; max 10 active per subteam (checked in service layer)
player_trainer_ref	player, kind (first, personal), trainer (nullable), free_first_name, free_last_name	Linked trainer or plain name, never both
player_status_event	player, status (active, dropped_out), effective_at, reason, set_by	History; player.status is the latest value

5.2 Competition structure

Table	Key fields	Notes
season	id, name, starts_on, ends_on, config_locked_at	Config frozen once started
tournament_settings	id, season, k_factor, table_count, rest_minutes, finals_priority, valid_from, created_by	Versioned rows, never updated in place; see ratings
tour	id, season, ordinal, name	Placement weight for the season score

Table	Key fields	Notes
stage	id, tour, ordinal, name, format (round_robin, single_elim, composite), config (JSON)	Config holds group count, qualifiers per group, bracket seeding map
stage_group	id, stage, name	Round-robin groups or bracket labels
stage_entry	id, stage, group, subteam, seed, seed_rating	seed_rating is a snapshot at draw time
standing	stage, group, subteam, played, won, lost, game_diff, set_diff, point_diff, place	Recomputed on match finish
tour_placement	tour, subteam, place	Input to season score
season_standing	season, subteam, score, rank, tiebreak_note	Ascending by score, head-to-head tie-break

5.3 Matches, lineups and scoring

Table	Key fields	Notes
team_match	id, stage, group, round, subteam_a, subteam_b, status (scheduled, called, in_progress, finished), hall_table, scheduled_at, called_at, started_at, finished_at, winner, result_type (played, no_show, forfeit)	Ends at 3 game wins
lineup	id, team_match, subteam, submitted_by, submitted_at, locked_at	Unique (match, subteam); visibility rule enforced in queries
lineup_slot	lineup, slot (1 to 5), player (nullable)	Slot gender fixed by slot number; null means auto-loss
lineup_replacement	team_match, subteam, slot, old_player, new_player, made_by, reason, made_at	Main Judge or Organizer only
singles_game	id, team_match, slot, player_a, player_b, status (pending, in_progress, finished, cancelled), winner, sets_a, sets_b, auto_loss	Cancelled when the match is already decided
game_set	id, singles_game, number (1 to 5), points_a, points_b, first_server, winner	Derived values cached for fast reads

Table	Key fields	Notes
point_event	id, game_set, seq, kind (point, undo_point, undo_set), scorer, server, score_after, created_by, created_at	Append-only log; undo is a new event, the score is rebuilt from it

5.4 Ratings, hall and audit

Table	Key fields	Notes
rating_history	id, player, singles_game (nullable), rating_before, rating_after, k_factor, source (initial, import, game), created_at	Unique (player, singles_game); the K used is stored, so later K changes never rewrite history
rating_import	id, filename, imported_by, imported_at, row_count	For the later CSV import; rows land in rating_history with source import
hall_table	id, number, label, is_active	12 rows by default
table_assignment	id, team_match, hall_table, planned_start, assigned_by, mode (auto, manual)	Overlap and rest-time checks in the allocator
line_judge_assignment	line_judge, hall_table, day	Unique (line_judge, day); judge scores only that table that day
audit_log	id, actor, actor_role, action, entity_type, entity_id, before (JSON), after (JSON), created_at, ip	Insert-only: no update or delete privileges for the app role
translation (optional)	entity_type, entity_id, field, locale, value	Only if team and region names need Kazakh and Russian forms

5.5 Invariants enforced by the database

- Unique (subteam, team, age_group), unique (lineup per match and subteam), unique (rating_history per player and game).
- Check constraints: set points and winner consistent with 11-point, win-by-2; slot between 1 and 5; match cannot be finished with fewer than 3 game wins unless forfeit or no_show.
- Foreign keys everywhere; soft deletes only for players and accounts (history must stay readable).
- Lineup lock: updates to lineup_slot rejected once locked_at is set, except through the replacement path.
- Indexes for the hot paths: point_event (game_set, seq), team_match (status, scheduled_at), team_match (hall_table, status), player (subteam, status).

6. Real-time scoring design

The server is the single source of truth: a point appears on any screen only after the database transaction that recorded it has committed. Tablets do not update the score optimistically.

6.1 Flow of one point

1. The line judge taps a button; the tablet sends a command over its WebSocket with a client-generated idempotency key.
2. The consumer checks role, table assignment, match state and that the game is in progress.
3. In one transaction: append a `point_event`, update the set and game, and if a set, game or match ends, update statuses, run the Elo update, and write the audit row.
4. After commit, the server publishes one event to the Redis channel layer; Channels fans it out to every subscribed socket, and the tablet gets an acknowledgement.
5. A duplicate command with the same idempotency key returns the original result and changes nothing.

6.2 Channels and messages

Group	Subscribers	Content
`match.<id>`	Public viewers, judges	Score, server indicator, set and game results for one match
`board.live`	Public home and scoreboard pages	Compact status of all in-progress and called matches
`table.<n>`	Line judge tablet (read and write)	Full state of the current match on that table, plus commands
`hall.control`	Organizer, Main Judge	Status changes, allocations, lineup submitted flags
`team.<id>`	Trainer	Upcoming matches, lineup deadlines, reveal events

Every message carries the match `version`, a number that increases by one per change. A client that sees a gap fetches a fresh snapshot over REST and resumes from it. Lineups are never published before both sides submit; the reveal is one event delivered to everyone at the same moment.

6.3 Connection behaviour

- Heartbeat every 20 seconds; the client reconnects with exponential backoff and jitter, then resubscribes and resyncs from a snapshot.
- The tablet shows a clear disconnected state and disables scoring buttons when the socket is down (the brief requires a working network connection).
- Public pages fall back to polling a cached REST endpoint every 5 seconds if WebSocket cannot connect.
- WebSocket authentication uses the session; origin is checked; commands are rate-limited per connection.

6.4 Proposed latency targets

These are proposals for sign-off, measured at the server edge and on a typical venue Wi-Fi or mobile connection.

Metric	Target (p95)	Hard limit (p99)
Command received to transaction committed	100 ms	250 ms
Button tap to acknowledgement on the same tablet	300 ms	600 ms
Button tap to update visible on a viewer's screen	1 s	2 s
WebSocket connect and first snapshot	1 s	3 s
Reconnect and resync after a dropped connection	3 s	8 s
REST read of a cached public page	150 ms	400 ms

Load assumptions. About 15 matches at once, one point roughly every 10 to 15 seconds each, gives around 1 to 1.5 write events per second. With 1,000 viewers on the hottest streams, fan-out is on the order of a thousand messages per event, which a single ASGI node and Redis handle comfortably. These are estimates; the acceptance test in section 7 checks them.

7. Non-functional requirements

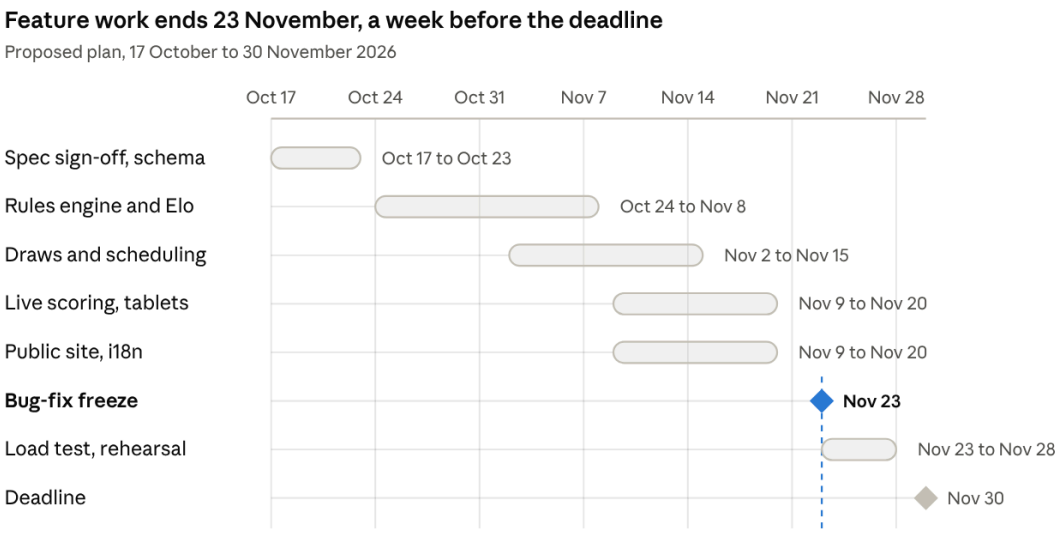
Area	Requirement
Capacity	1,000 concurrent viewers and 15 concurrent matches as the target; the acceptance test runs at 2x (2,000 viewers, 30 matches) with the latency targets of section 6.4 still met
Availability	Best effort outside event days; on tournament days an uptime target of 99.9% for scoring and public pages, with an uptime monitor alerting the team
Backups	Nightly full backup plus continuous WAL archiving to off-site, encrypted object storage; retention 14 daily and 8 weekly; recovery point objective 5 minutes, recovery time objective 30 minutes
Restore proof	A restore to a clean machine is rehearsed before launch and repeated on a schedule; a backup never restored is not counted as a backup
Languages	Russian, Kazakh; all UI text externalised; language in the URL (`/ru`, `/kk`); Russian as default and fallback for a missing Kazakh string; fonts and collation must handle Kazakh Cyrillic letters; dates and numbers localised
Security	HTTPS only; secure, HTTP-only session cookies; CSRF protection; Argon2 password hashing; login rate limiting; role checks on every endpoint and socket; audit log insert-only for the application database role

Area	Requirement
Personal data	The roster includes minors. The public site shows name, team, age group, rating and photo; date of birth and trainer contact data stay private. Photos are published only with consent captured by the trainer
Auditability	Every score change, undo, lineup submit, replacement, manual table override and settings change is logged with actor, time, before and after
Data integrity	Scores, game results, ratings and statuses change in one transaction; ratings never recomputed retroactively when the K-factor changes
Devices	Judge tablets in landscape with large touch targets; public pages mobile-first; a large-format scoreboard view for hall screens
Observability	Error tracking, structured logs, metrics for socket count, message latency, queue lag, database and disk; alerts routed to the team during events
Testing	Unit tests for set and match scoring, serve rotation, Elo, seeding, composite stage advancement and tie-breaks; integration tests for lineup and replacement rules; a load test; and a full dress rehearsal that simulates a tournament day on the real VPS
Delivery	CI on every push, database migrations reviewed, one-command deploy, a staging copy of the VPS setup

Acceptance test for go-live. All of the following pass on the production-like VPS: simulated 72-subteam draw and schedule generated without conflicts; 30 simulated matches scored through real tablets for 30 minutes with 2,000 simulated viewers; the p95 and p99 targets met; a forced network drop recovers within the stated time; a database restore from backup succeeds.

8. Milestones, risks and open questions

The plan below fits the 6 weeks from 17 October to 30 November with a bug-fix freeze on 23 November, so the last week is rehearsal and repair, not new features. Dates are proposals to be agreed with the team.



milestone timeline · 17 Oct to 30 Nov 2026

Build runs in overlapping streams until the freeze on 23 November; the final week is load testing, a full dress rehearsal and repairs.

8.1 Risks

Risk	Impact	Mitigation
Only 6 weeks for 3 engineers	Features cut late	Fix scope now; the CSV import, hall-screen polish and Kazakh copy review are the first to defer
Composite stage formats are open-ended	Draw and advancement logic balloons	Support the formats actually used this season as data-driven templates; add others later
Formulas missing from the brief (Elo, season score)	Rework in rating and ranking code	Confirm in open questions 1 and 2 this week
Venue network failure	Scoring stops (tablets need a network)	Test the venue network in advance, bring a backup uplink; let the Main Judge enter scores from any device
Load behaves differently on the real VPS	Slow live updates in front of 1,000 viewers	Load test at 2x on the production-like VPS by week 5
Late translations	Untranslated screens at launch	Freeze UI strings by 9 November; Russian as fallback

Risk	Impact	Mitigation
Minors' personal data	Legal and reputational exposure	Limit public fields, capture photo consent, check hosting rules (open question 7)

8.2 Open questions

1. **Elo details.** The source spec leaves the formula blank. Confirm standard Elo with the 400 scale, and that automatic-loss games leave ratings unchanged.
2. **Season score.** The source spec leaves the formula blank. Confirm the sum of tour placements, and whether tours are weighted.
3. **Seeding input.** Lineups exist only per match, so confirm that seeding uses each subteam's top 5 eligible players (3 boys, 2 girls) at a freeze time.
4. **Formats this season.** Group sizes, qualifiers per group, bracket layouts, third-place matches and in-group tie-breaks.
5. **Playing up.** Age category is by birth year only; confirm that no player may play in an older age group.
6. **Public SEO.** Do search results and link previews matter for public pages? If yes, add server-rendered meta tags or move the public site to Next.js.
7. **Hosting and privacy.** VPS country and the consent process for minors' data under Kazakhstan's personal data rules; confirm with counsel.
8. **Judge fallback.** If a tablet fails mid-match, may the Main Judge score that table? Proposed: yes.
9. **Hall rules.** Default rest time between a player's games, and how finals priority is ranked against rest time.
10. **Tablets.** Which devices, and what network and backup uplink the venue has.